VIT

[VIT LOGO — paste the official logo here]

Vellore Institute of Technology

PROJECT REPORT

CC: [COURSE CODE]

CN: [COURSE NAME]

Title: OS-Tutor-RAG: A Retrieval-Augmented, Socratic Tutoring System for the Operating Systems Course, with a Measured Evaluation of Its Pipeline

Submitted by

Adhyan Jain (24BCI0111)
Atharva Sheersh Pandey (24BCI0069)

Faculty: [FACULTY NAME] Slot: [SLOT] Semester: [SEMESTER]

October 2026

Table of Contents

1. Introduction.....3
2. Literature Review.....3
3. Problem Statement.....4
4. Objectives.....5
5. Methodology.....5
6. Results and Discussion.....10
7. Conclusion.....15
8. Limitations.....16
9. Future Work.....16
10. References.....17

Principal subsections

5.1 System overview.....5
5.2 Ingestion and chunking.....6
5.3 Retrieval and context expansion.....6
5.4 Generation: the prompts.....7
5.5 The tutoring layer.....8
5.6 Evaluation protocol.....9
6.1 Retrieval and defects.....10
6.2 Prompt results.....12
6.3 Noise floor.....13
6.4 The tutoring layer.....13
6.5 Extended research: RCR.....15

1. Introduction

Students in an introductory Operating Systems (OS) course meet a dense vocabulary almost at once: processes, system calls, context switches, paging, locks, semaphores, deadlock. Lecture decks are terse by design, and a textbook chapter assumes a reader who already knows what to look for. A student who is stuck at eleven at night has two usual options: a general-purpose chatbot, which answers fluently but is not tied to the course and sometimes invents details, or a search through slides that never explains anything.

Retrieval-Augmented Generation (RAG) addresses the first problem by making a language model answer from retrieved passages instead of from memory alone. It does not, by itself, make the system a good tutor. A tutor does more than answer. It notices what the student already understands, asks a short question to check, gives a hint before the answer, corrects a specific wrong belief gently, stays patient with a student who says "I don't get it", and, for a student who does not know what to ask, suggests sensible next questions.

This project, OS-Tutor-RAG (called "OS Tutor" in the interface), builds both halves. The first half is a RAG pipeline over the course's own lecture decks, OSTEP textbook chapters and lab handouts, built as a controlled comparison so that every design choice (chunking, retrieval technique, reranking, diversification, prompting) is backed by a measurement rather than by convention. The second half is a tutoring layer built on one central rule: pedagogical decisions live in tested code and the language model only phrases what the code has already decided.

A second, related research track studies how reference choice affects the reliability of executable OS-reasoning benchmarks (Reference-Choice Robustness, RCR). It is summarised in Section 6.5 as an extended contribution.

2. Literature Review

Five works shape the design. Each is summarised with the decision it informed here.

2.1 Retrieval-augmented generation

Lewis et al. (2020) introduced RAG: a generator conditioned on passages retrieved from an external index, so that knowledge can be updated by editing the index and answers can point to sources. This is the architectural premise of OS Tutor. The paper also frames the retrieval and generation components as separately measurable, which motivated our split between retrieval-side and generation-side metrics.

2.2 Dense passage retrieval

Karpukhin et al. (2020) showed that learned dense embeddings of questions and passages can outperform BM25 on open-domain question answering, because they match meaning rather than words. Our retrieval comparison tests this claim on a small, single-course corpus; we use

the BGE-large embedding model (Xiao et al., 2023) and find dense retrieval reaches recall@5 of 1.000 while BM25 reaches 0.885 on the corrected benchmark.

2.3 Query transformation: HyDE

Gao et al. (2022) proposed Hypothetical Document Embeddings: ask a model to write a plausible answer, embed that, and retrieve with it. The intuition is that an answer sits closer to answer-bearing passages than the question does. We implement HyDE and a multi-query variant and measure them; HyDE improves retrieval breadth (full@5 0.692 against 0.577 for dense) but the gain does not survive to answer quality.

2.4 Automated RAG evaluation: RAGAS

Es et al. (2023) defined reference-free and reference-based metrics for RAG (faithfulness, answer relevancy, context precision and recall) computed with an LLM judge. We use RAGAS with a local judge. Our own findings add a caution that the paper's framework makes easy to miss: faithfulness rewards refusal, so a system that refuses everything scores 1.0.

2.5 LLM-as-a-judge

Zheng et al. (2023) examined using strong LLMs to grade LLM outputs and catalogued their biases, including self-preference and verbosity. We took one direct design consequence: the judge (qwen2.5:7b) is from a different model family than the generator (llama3), to avoid grading a model's own style favourably. Our 7B judge is far weaker than the judges that paper studies, which we state as a limitation.

2.6 Tutoring and scaffolding

The instructional design follows the long-standing notion of scaffolding (Wood, Bruner and Ross, 1976): support that is withdrawn as the learner gains competence. In the tutoring layer this appears as a ladder of hint, then a narrower scaffold question, then a concise explanation after repeated misses, and as a concept state that moves from UNKNOWN towards MASTERED.

3. Problem Statement

Existing options for a student learning OS fail in complementary ways. Slides and textbooks are authoritative but passive. General chatbots are interactive but ungrounded, so a student cannot tell a correct explanation from a plausible fabrication. A plain RAG chatbot is grounded but still behaves like a search box: it answers the question asked, says nothing about what to ask next, and gives a student who does not know the subject no way in.

The problem addressed is therefore two-fold. (1) Build a RAG pipeline over OS course material whose configuration is justified by measurement, including the measurement of the evaluation method itself. (2) Make the chatbot behave as a Socratic tutor that is friendly,

interactive, checks understanding with graded questions, adapts when the student struggles, and proposes good follow-up questions even when the student knows nothing about the topic, while keeping every answer grounded in the course material.

4. Objectives

- **O1.** Ingest and index the course material (PPTX decks, PDF chapters, DOCX handouts) without silently losing content such as tables or legacy-format files.
- **O2.** Compare chunking, retrieval, reranking, diversification and prompting strategies against a hand-built golden question set, and adopt only what the evidence supports.
- **O3.** Quantify the reliability of the evaluation itself (run-to-run noise, metric blind spots) so that conclusions are not drawn from differences below the noise.
- **O4.** Provide a streaming web chatbot with three explanation levels (ELI5, undergraduate, exam preparation) and source citations.
- **O5.** Implement a tutoring layer: 30 authored concepts with 96 graded questions, a concept-state machine, a hint, scaffold, explain-after-two-misses ladder, 16 misconception probes, a three-question quiz mode, difficulty detection and fixed, safe replies to distress, off-topic and prompt-injection messages.
- **O6.** Offer an orientation reply, a learning path, starter chips and three layers of follow-up suggestions so that a student who knows nothing can still move forward.
- **O7.** Keep the work reproducible and honest about limits: automated tests for the tutoring policy (877 tutor tests plus 427 pre-existing), recorded refutations, and stated caveats.

5. Methodology

5.1 System overview

Figure 1 shows the system. An offline stage turns the course files into a searchable index. An online stage retrieves passages for each question, expands them to their parent slide or page, builds a teaching prompt and streams the model's answer to the browser over Server-Sent Events (SSE). The tutoring layer wraps every turn in two half-steps. plan_turn runs before retrieval and either answers entirely with authored text chosen by code (triage, orientation, a graded reply to the pending question, a hint, a quiz question), costing no retrieval and usually no model call, or hands the message to the RAG pipeline as a plan. finish_rag runs after the grounded answer and appends at most one Socratic question and the follow-up chips. The server streams three kinds of event over SSE: token (answer text), ui (chips, question card, concept state) and sources.

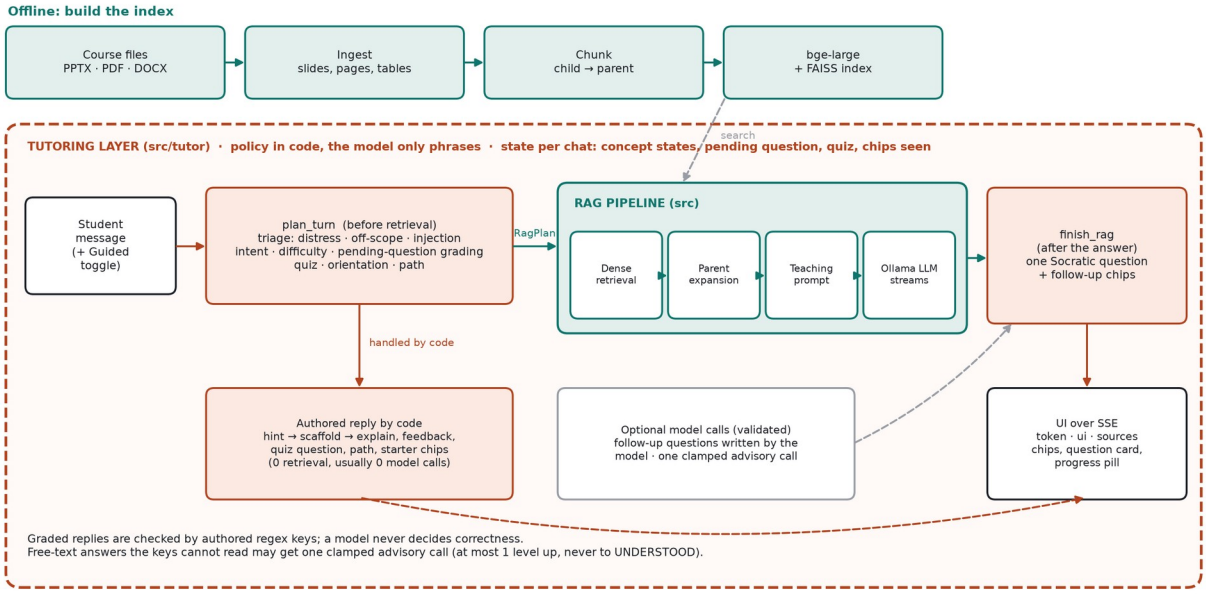


Figure 1. Architecture of OS-Tutor-RAG: the tutoring layer (red) wraps the RAG pipeline (teal) on every turn.

5.2 Ingestion and chunking

The corpus used for the measurements in this report was 22 extracted documents from 10 source files (5 PPTX decks, 4 OSTEP PDF chapters and one DOCX of 13 sections) covering processes and shell programming. The live system's corpus has since grown: all 36 indexed files in data/raw (adding memory management, concurrency, deadlock and IPC material) are searchable; the measurements were not repeated on it. Extractors read each slide's title, bullets and table cells, each PDF page, and each DOCX heading section.

Chunking is matched to the format rather than treated as a tunable axis. PPTX uses structure-aware chunking: each bullet becomes a small child chunk prefixed with its slide title, and the whole slide is stored as its parent. PDF uses page-aware semantic chunking bounded by the page, with the page as the parent. DOCX uses overlapping windows with the section as parent. The child/parent design ("small-to-big") matches on narrow passages and generates from their full context.

5.3 Retrieval and context expansion

Children are embedded with BAAI/bge-large-en-v1.5 and indexed with FAISS. The default retriever is dense search. Implemented alternatives are BM25, hybrid Reciprocal Rank Fusion (RRF), HyDE, and a multi-query wrapper that generates reformulations, retrieves for each and fuses by RRF. Optional rerankers (a cross-encoder, bge-reranker-large, and a pointwise LLM scorer) and Maximal Marginal Relevance (MMR) diversification are implemented and were measured, then turned off by default because they did not help. After selection, each child is replaced by its parent text capped at 400 tokens and windowed around the match when larger, so that matching stays precise while the model reads complete context.

5.4 Generation: the prompts

Generation runs on a local Ollama server (llama3 by default) at temperature 0. Two prompts matter and they are different things. The evaluation prompt ("few_shot") is the one the RAGAS experiments scored. The teaching prompt is what students actually receive in the app, and it is NOT the few_shot prompt: it was written for explaining to a beginner and for leaving the questioning to the tutoring layer, and no RAGAS metric has been computed for it. Both are reproduced exactly.

Evaluation prompt ("few_shot", the measured default):

```
Answer the question using only the context below. If the context
doesn't contain the answer, say so.

Answer in continuous prose, stating each fact once. Do not add preamble
about the context.

Example question: What does the OS do when it creates a process?
Example answer: The OS allocates memory and creates the memory image,
loads the code and data from the executable on disk, creates the runtime
stack and heap, and opens basic files.

Example question: What are user mode and kernel mode?
Example answer: CPU hardware has multiple privilege levels: user mode
runs user code, and kernel mode runs OS code such as system calls.

Context:
{context}

Question: {query}

Answer:
```

Teaching prompt (undergraduate level; ELI5 and exam-prep differ in the persona paragraph):

```
You are a knowledgeable teaching assistant for an undergraduate
operating systems course, working from the context below. Assume the
student knows basic CS fundamentals (variables, functions, memory, the
CPU) and use standard OS terminology, with an analogy only where it
clarifies a genuinely tricky point.

{instructions}      <- the guided block below, or the direct block
Ground everything in the context below. If the context does not cover
part of the question, say so plainly rather than filling the gap from
outside knowledge.

{history_block}{misconception_block}{continuation_block}
Context:
{context}

Student's message: {query}

Your turn:
```

Guided instructions (default; 'Guided questions' switched on), abridged:

```
Tone: you are a patient, friendly tutor, like a helpful senior guiding a
junior. Be warm and encouraging without gushing; never condescending.
```

```
Assume the student may know nothing about the topic: define any term a
beginner would not know the first time you use it, and keep sentences short.

How to answer:
- Answer what the student actually asked, directly and completely, from the
  context. Explain first; never withhold the explanation to make the student
  guess. A separate step of the app adds one short check-your-understanding
  question after your answer, so do NOT end with a question of your own and
  do NOT quiz the student.
- Keep it focused: the idea, why it matters, and one concrete example or
  analogy when it helps.
- If a note below says the student may hold a misconception, address it
  gently inside your explanation without making them feel foolish.
- If the message is a short continuation ("continue", "go on"), continue
  naturally from exactly where the conversation left off.
```

With the Guided questions toggle off, a second block is used instead: answer completely and ask the student no questions. The misconception note that the prompt carries now comes from deterministic patterns matched against the student's question (16 authored misconceptions), which replaced a per-message model call that classified the question; this removes one blocking model call and its latency from every turn. The old call remains behind an environment flag as a fallback.

Three other prompts exist and were measured: "strict" (answer or say the context lacks the answer), "part_coverage" (answer each part the context supports) and "cot" (a labelled Reasoning section before the Answer, stripped before scoring).

5.5 The tutoring layer

The layer (src/tutor) follows one guiding rule: the model never decides whether an answer is right; policy lives in tested code and a model may only phrase an outcome. The state of one conversation (concept states, the pending question, the quiz, the chips already shown) is kept per chat. Its parts are:

- **Knowledge model:** authored course knowledge: 30 concepts in five units (Foundations, Processes, Memory, Concurrency, IPC and tools), each with detection aliases, prerequisites, a description and an importance; 96 graded questions; 16 misconceptions. The longest matching alias wins, so "page fault" maps to demand paging rather than to the generic "process". Each question's model answer is itself tested to pass its own answer key, and a hint must not be parroted back as an answer.
- **Triage:** every message first passes a deterministic filter for distress ("I'm going to fail"), clearly off-topic requests and prompt-injection attempts ("ignore your previous instructions"). Each gets a fixed, kind reply that does not depend on a model being reachable; a distressed student is encouraged to talk to the instructor and offered one small next step, not quizzed.
- **Intent and difficulty:** deterministic intent detection (hint, just tell me, example, skip, pause, quiz, path, lost, greeting, continue, prerequisites, what next) and difficulty detection (confused, frustrated, overwhelmed, asking to repeat, uncertain). A struggling

student first gets a simpler explanation at ELI5 level, then, if it still has not clicked, the full explanation. A firm new question while a question is pending ("Compare FCFS and SJF") is never graded as an attempt.

- **Concept state machine:** each concept moves through UNKNOWN, INTRODUCED, ATTEMPTED, PARTIALLY_UNDERSTOOD, UNDERSTOOD and MASTERED. Every change goes through one function that records its source. There is no wall clock: a turn counter implements the cooldown, so behaviour is reproducible and testable. MASTERED is reachable only from UNDERSTOOD through a correct transfer question.
- **Grading and the hint ladder:** a correct or partial answer is judged by regex answer keys (CORRECT, PARTIAL, MISCONCEPTION, UNCLEAR). Feedback is honest about which: a partial answer is told what is right and gets the scaffold; a misconception gets a probing question first and the correction only after two misses; after two non-correct attempts the tutor stops asking and explains. A correct answer earns, in quiz mode, a harder question at the next level of the ladder (recall, understanding, application, analysis).
- **One Socratic question per answer:** after the grounded answer, at most one short question about the concept just discussed (a new concept starts with recall, a partial one with interpretation, an understood one with transfer), never repeating a question and never re-testing an understood concept, and never stacking a second question on a pending one. A Guided questions switch in the UI turns this off for plain answers.
- **Quiz mode:** "test my understanding" starts three questions on one concept, then a short summary of how the student did.
- **Orientation and follow-up suggestions:** for a student who knows nothing: a greeting or "I'm new" returns a short learning path built from the foundations, "I don't know what to ask" returns an orientation with starter chips, and the empty chat shows a course map. Follow-up chips come from three layers: authored, concept-aware chips (explain with an example, test my understanding, what to know first, how it connects to the next idea); chips built from the slide titles of the sources the answer was grounded in, which work for any topic in the corpus; and follow-up questions written by the model, validated by code before being shown. Every chip is a complete message, so tapping it is the same as typing it.
- **Where a model may help:** two optional, fail-soft model calls. One phrases a reply to a free-text answer the answer keys cannot read and proposes a classification, which is clamped: at most one level up, never to UNDERSTOOD, never down. The other writes follow-up questions. A question written by the model has no answer key, so it is never graded and moves no concept state.

5.6 Evaluation protocol

The pipeline is scored with RAGAS on a golden set of 26 questions (5-6 per topic: OS basics, process abstraction, process API, process execution mechanism, shell programming). Each

question is paired with a ground-truth answer and the slide or page it is answerable from (source_parent_ids), and is worded unlike the slide title so a retriever cannot win by matching the heading. For each question the pipeline retrieves contexts and generates an answer; RAGAS then sends the judge model the question, the retrieved contexts, the generated answer and the ground truth, and the judge returns claim-level decisions from which seven metrics are computed.

Table 1. The seven RAGAS metrics.

Metric	Judges	How it is computed (inputs to the judge)
context_precision	retrieval	Fraction of retrieved contexts judged useful for the question (question, contexts, ground truth)
context_recall	retrieval	Fraction of ground-truth claims supported by the contexts
context_entity_recall	retrieval	Overlap of entities extracted from ground truth and from contexts (found unreliable here)
faithfulness	generation	Answer split into claims; fraction entailed by the contexts (hallucination guard)
answer_relevancy	generation	Questions regenerated from the answer, embedded and compared with the real question
answer_similarity	generation	Embedding cosine between answer and ground truth
answer_correctness	whole pipeline	Weighted blend of answer_similarity and an F1 over answer vs ground-truth statements

Generation and judging are run in two passes (all answers first, then all scoring) because the generator and judge cannot share the 8 GB GPU. Retrieval is also measured directly, without any model in the loop, with recall@k, MRR and full@k (all required source parents present) against source_parent_ids. Costs quoted in USD are hypothetical, priced from token counts at published API rates; no paid API was called.

6. Results and Discussion

6.1 Retrieval and defects

The most important early result was negative for the project's original premise. A chunking defect, slide titles indexed as standalone chunks, made BM25 look dramatically worse than dense retrieval (answer_correctness 0.509 against 0.616), because twelve of 81 slides have question-shaped titles that match a user's question lexically while containing no answer. After the fix, and after parent expansion (generation had been receiving only the ~50-character child), refusals fell from 11.7% to about 0%. Fixing a defect was worth more than choosing among techniques.

Table 2. Retrieval metrics on the corrected golden set (26 questions).

Technique	R@1	R@3	R@5	R@10	full@5	MRR
dense	0.769	0.962	1.000	1.000	0.577	0.875
hyde	0.654	0.923	1.000	1.000	0.692	0.804
dense + multi_query	0.808	0.923	0.962	1.000	0.577	0.873
hybrid_rrf + multi_query	0.769	0.885	0.962	0.962	0.654	0.848
hybrid_rrf	0.769	0.808	0.923	1.000	0.538	0.826
bm25	0.654	0.731	0.885	0.923	0.462	0.736

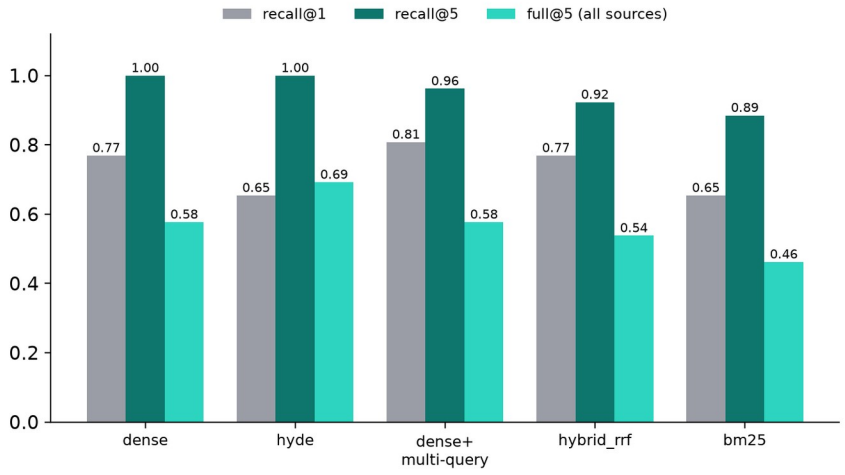


Figure 2. Dense retrieval saturates recall@5; HyDE leads full@5; no technique dominates.

Dense retrieval is the best all-rounder and the only technique needing no model call. HyDE buys breadth (full@5) at the cost of precision (R@1); multi-query buys precision but dilutes recall. A second benchmark effect is worth recording: the benchmark itself was biased against lexical retrieval. Rewriting questions to avoid the slide's wording raised BM25's recall@5 from 0.500 to 0.885 with no change to BM25, so the original claim that BM25 fails half the time measured the question set.

Post-retrieval refinements all measured worse than leaving them out:

Table 3. Refinement stages.

Stage	Measured effect
Cross-encoder reranking	-0.05 answer_correctness on the biased benchmark; +0.013 after correction (within noise)
MMR diversification	-0.038 hit@8 (drops a correct slide)
Per-parent capping	no effect on hit rate
BM25 fusion in hybrid_rrf	-0.077 recall@5 against dense alone

Eight-variant sweep (26 questions, 5 query variants where multi-query is on):

Table 4. Focused sweep. The spread across all eight is 0.052; cost and quality were anti-correlated.

Variant	composite	ans_corr	faithful.	ctx_recall	LLM calls
dense + multi_query	0.772	0.662	0.958	0.936	480
hyde + cross_encoder	0.764	0.644	0.958	0.978	491
hyde	0.752	0.631	0.929	0.958	510
dense + cross_encoder	0.744	0.642	0.949	0.926	448
hyde + multi_query + cross_enc.	0.743	0.656	0.882	0.968	593
dense	0.740	0.629	0.899	0.942	451
hyde + multi_query	0.734	0.637	0.912	0.960	615
dense + multi_query + cross_enc.	0.720	0.607	0.883	0.936	471

6.2 Prompt results

Against the identical best configuration, two worked examples in the prompt raised answer_correctness more than the whole retrieval sweep. The diagnosis was that answers lost points to statement granularity (a content-identical answer scored 0.56 because it split one reference statement into two bullets), since answer_correctness compares sets of statements.

Table 5. Prompt comparison on dense + multi_query.

Prompt	composite	ans_corr	faithfulness	ctx_recall
strict (previous default)	0.763	0.664	0.942	0.946
few_shot (adopted)	0.784	0.756	0.986	0.928
part_coverage	0.712	0.631	0.819	0.936

Chain-of-thought was the next hypothesis and was refuted: answer_correctness fell from 0.7478 to 0.6887 and faithfulness from 0.981 to 0.933, because the reasoning step shortened answers by 27% (26.0 against 35.6 words), and the lost words were lost statements. The part_coverage prompt invited the model to fill gaps from its own knowledge and dropped faithfulness to 0.819. Both are kept in the code as recorded negatives.

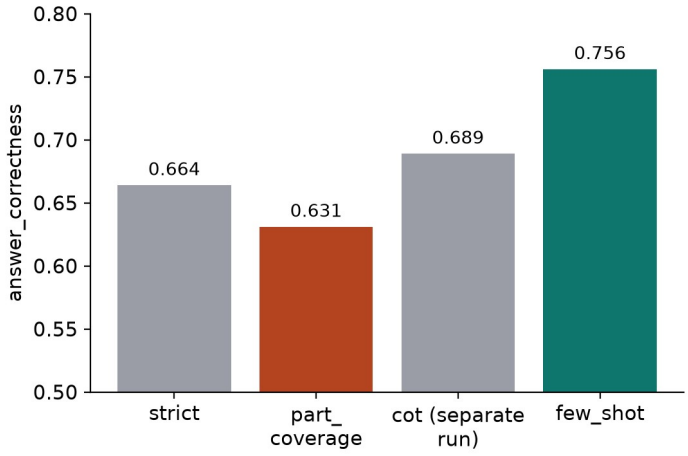


Figure 3. Prompt choice (cot was scored in a separate run, separate run: few_shot 0.748, cot 0.689).

6.3 Noise floor and what can be believed

The same configuration (dense + multi_query + cross-encoder, few_shot prompt) was scored twice and gave 0.7752 and 0.7478. The run-to-run noise floor on answer_correctness is therefore about 0.027, from stochastic multi-query variant generation and from nondeterministic numerics in the inference server that persist even at temperature 0. Figure 4 reads every result against it.

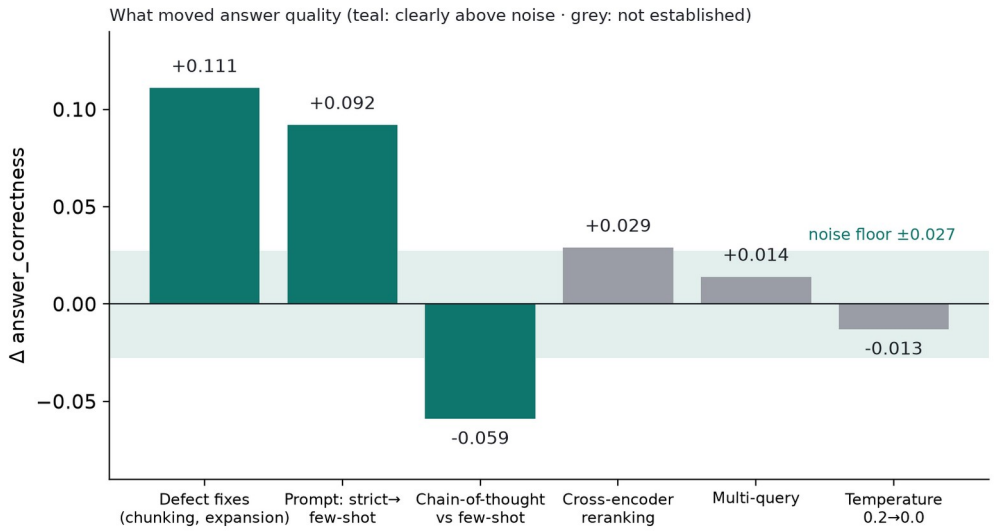


Figure 4. Effect of each change on answer_correctness against the noise floor.

Only the defect fixes, the few-shot prompt and the chain-of-thought regression are established. The reranking gain (+0.029) and the multi-query gain (+0.014) are not: the retrieval-technique comparison that occupies most of the effort lives inside its own error bars. Two further cautions came from the metrics. Faithfulness rewards refusal (a pipeline that refuses everything scores 1.0), so refusal rate was tracked beside it; and context_entity_recall was shown to be extraction disagreement rather than missing information (seven ground-truth entities, all present in the context, matched zero).

6.4 The tutoring layer

The tutoring layer's decisions are deterministic, so it is evaluated by tests and live runs rather than by a judge model. The tutor suite (tests/tutor) has 877 passing tests, alongside 427 pre-existing tests that still pass. They check, among other things: that every question's model answer passes its own key and is classified CORRECT; that partial, misconception and unclear inputs are graded as such; that understanding is never regressed by a partial or unclear answer; that the advisory suggestion is clamped; that a question is not repeated and an understood concept is not re-tested; that a hint repeated back as an answer is not accepted; that distress, off-topic and injection messages get the fixed reply while no genuine course question trips triage; that every chip is a self-contained message that routes correctly and that chips are not repeated; that a firm new question mid-quiz is answered rather than graded; and

an event-contract test of the API with retrieval and the model stubbed. A scripted dialogue (test_new_student_dialogue) follows a student who says "I'm new to operating systems" through the path, a tapped suggestion, the grounded answer and exactly one appended question. The system was also run end to end through the web interface with llama3.1:8b and mistral: starter, learning path, explanation, Socratic question, graded reply, hint chips and the progress pill.

Figure 5 is real output of the engine (plan_turn and finish_rag) for a new student. The grounded answers are written by the model at run time, so a placeholder stands in for them; every tutor line below, including the path, the question, the scaffold, the grading feedback, the chips and the quiz opener, is produced by the code. Long lines are abridged with [...]; the driver is submission/build/transcript.py.

Student: I'm new to operating systems

Tutor: Let's build up from the foundations. I'd start here:

- Operating system: The software layer between user programs and the hardware. - User mode and kernel mode: The CPU has privilege levels. - System calls and traps: A system call is how a user program asks the OS to do something [...]

Tap one to begin, or ask in your own words. No silly questions here.

chips: Explain operating system in simple words | Explain user mode and kernel mode in simple words | Explain system calls and traps in simple words

Student: Explain operating system in simple words

Tutor: [grounded answer streamed by the model from the retrieved slides]

Think about this: Why do we put an OS between programs and the hardware, instead of letting each program talk to the hardware directly?

chips: Give me a hint | Just tell me | Skip this question

Student: no idea

Tutor: That's completely fine, you don't need to know this already.

Operating system, in simple terms: The software layer between user programs and the hardware.

An easier way in: Imagine ten programs all writing to the same disk at once. What would you want in the middle to keep that sane, and to make it easier to program?

chips: Just tell me | Skip this question

Student: hint

Tutor: That is the only hint I have for this one. Have a go, or say "just tell me" and I will show the idea.

chips: Just tell me | Skip this question

Student: It hides the hardware behind a simple abstraction, and lets many programs share it safely without interfering.

Tutor: Exactly. The OS gives programs a simple abstraction of the hardware (convenience) and makes sure many programs can share it safely and efficiently (isolation and efficiency).

chips: Explain operating system with a simple example | Test my understanding of operating system | How does operating system connect to user mode and kernel mode?

Student: I'm so confused by threads

Tutor: That's okay, this one can take a moment to click.

[grounded answer streamed by the model from the retrieved slides]

chips: Explain threads more simply | Explain threads with a simple example | Where should I start?

Student: Test my understanding of paging

Tutor: Let's check your understanding of Paging: 3 short questions. Answer in your own words; there are hints if you want them.

Question 1 of 3: What does paging divide virtual memory and physical memory into?

chips: Give me a hint | Just tell me | Skip this question

Figure 5. Transcript of a new student with the real tutoring engine (abridged).

Two things the transcript shows. First, when the student says "no idea", the tutor does not mark an answer wrong: it normalises the uncertainty, restates the concept simply and offers an easier way in. Second, the tutor accepts a correct answer written in the student's own words, not only the textbook sentence, and then offers next steps that include a test of understanding.

This is verification of intended behaviour, not evidence of learning gains. No study with students was run, and there is no RAGAS-style metric for pedagogy: the RAGAS results in 6.1-6.3 measure answer quality for the few_shot prompt, not the teaching prompt or the tutor's questions. The regex answer keys can miss a correct paraphrase; the clamped advisory call mitigates this but does not remove it.

6.5 Extended research: Reference-Choice Robustness (RCR)

In parallel, the repository holds a formal study of a benchmark-design question that arose from evaluating models on executable OS tasks (scheduling, synchronisation interleavings, Banker's algorithm). When many valid trajectories exist for a task, does it matter which one is chosen as the gold reference? Over 1,152 outputs from four sub-10B models on 24 tasks, 50,000 sampled reference vectors changed rankings substantially (Kendall tau-b 0.618 +/- 0.351) and flipped pairwise winners in 17.56% of draws, while a reference-independent executable validator gave a stable ranking by construction. Reference-matching evaluators missed the model differences the validator found in almost every draw, following an exact attenuation law $E[E2] = E3/|V(x)|$. The result is a post-hoc reanalysis of a pilot whose preregistered verdict was CONDITIONAL, on one 24-task benchmark; it is reported here as scoped, and the full manuscript is in the repository (paper/PAPER_FINAL.md).

7. Conclusion

OS-Tutor-RAG delivers a grounded, streaming OS tutor and a tutoring layer that makes it behave like one. On the engineering side the central lesson is that measurement overturned the plan: the gains came from repairing silent defects (titles indexed as chunks, parents never read, tables and a legacy file never indexed) and from the answer prompt, not from choosing a cleverer retrieval technique, and the technique differences sit inside a measured noise floor of 0.027. On the tutoring side, the system explains first and then asks one graded Socratic question; escalates from hint to scaffold to explanation; probes 16 common misconceptions; runs a three-question quiz; answers a struggling student with a simpler explanation; and offers

an orientation, a learning path, starter chips and three layers of follow-up suggestions so that a student who knows nothing can still ask a good next question. All decisions are made by tested code (877 tutor tests) and the model is used for phrasing. What is not yet shown is a learning gain: that needs a study with students.

Source code and evidence:
https://github.com/adhyan-jain/OS-Tutor-RAG

8. Limitations

- Small benchmark: 26 questions over a 22-document, 10-file corpus. Differences of a few hundredths are not resolvable.
- Weak judge: qwen2.5:7b is a local 7B model and was not validated against human grading; a human rubric was deliberately deferred.
- Single run per configuration, no significance testing; the one variance estimate (0.027) is large relative to the retrieval effects.
- The corpus has since grown to all 36 indexed files; none of the numbers in Section 6.1-6.3 were re-measured on it. The tutor-facing teaching prompt differs from the few_shot prompt that RAGAS scored, and was not scored.
- Metric weaknesses: context_entity_recall is unreliable on conceptual text and half of the completeness composite; faithfulness rewards refusal.
- Ground truth is less complete than the corpus, so a more complete correct answer can be penalised as extra statements.
- The tutoring layer is verified by unit and dialogue tests and live smoke runs only: no study with students and no RAGAS-style metric for pedagogy. Its 30 authored concepts cover the main course topics, not every question a student could ask; regex answer keys can miss paraphrases (the clamped advisory call mitigates this, not removes it).
- Local 7-9B models only (8 GB GPU): latency and answer quality are bounded by them; costs in USD are hypothetical.
- RCR results come from one 24-task benchmark and four small models.

9. Future Work

- **Re-measure on the full corpus.** Rebuild the golden set across all 36 indexed files with several correct source parents per question, and re-run the sweep with 3-5 repeats per configuration and paired significance tests.
- **Validate the judge.** Hand-label a sample and measure agreement with qwen2.5:7b, and test a stronger judge.
- **Query decomposition.** Test splitting two-part questions, since 17 of 26 questions ask two things.

- **Learning-gain study.** Run a small pre/post study with students comparing the plain RAG chatbot with the tutoring layer, and define a measurable pedagogy metric (for example concept-state gain per session).
- **Paraphrase-robust grading.** Measure how often the regex keys miss a correct paraphrase on student data, and test whether wider keys or the advisory call close the gap.
- **Wider concept coverage.** Author more concepts and questions beyond the current 30 and 96, and generate candidates with a model but review them by hand before they enter the policy.
- **Persistent learner profiles.** Concept state is held in memory per chat; store it across sessions so the learning path adapts over a semester.
- **RCR.** Extend the reference-robustness study to more OS task families and larger models, and release the validator as a benchmark-design tool.

10. References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems 33. arXiv:2005.11401.

[2] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W. (2020). Dense passage retrieval for open-domain question answering. Proceedings of EMNLP 2020. arXiv:2004.04906.

[3] Gao, L., Ma, X., Lin, J., Callan, J. (2022). Precise zero-shot dense retrieval without relevance labels (HyDE). arXiv:2212.10496; ACL 2023.

[4] Es, S., James, J., Espinosa-Anke, L., Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. arXiv:2309.15217; EACL 2024 (demonstrations).

[5] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685.

[6] Xiao, S., Liu, Z., Zhang, P., Muennighoff, N. (2023). C-Pack: Packed Resources For General Chinese Embeddings (BGE models). arXiv:2309.07597.

[7] Wood, D., Bruner, J. S., Ross, G. (1976). The role of tutoring in problem solving. Journal of Child Psychology and Psychiatry 17(2), 89-100.

[8] Arpaci-Dusseau, R. H., Arpaci-Dusseau, A. C. Operating Systems: Three Easy Pieces (OSTEP). Arpaci-Dusseau Books (course reading).

[9] Project repository: OS-Tutor-RAG, FINDINGS.md (defects F1-F16, analyses A1-A20) and paper/PAPER_FINAL.md (RCR). https://github.com/adhyan-jain/OS-Tutor-RAG

arXiv identifiers [1]-[6] were checked against arxiv.org on 2026-10-24; [7] (Wood, Bruner & Ross 1976) and [8] (OSTEP) are standard textbook/journal citations.